

DBMS – ORDER MANAGEMENT SYSTEM

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 4

Objective:

1. Design Orders and Order_Details tables.
2. Manage customer product orders.
3. Store order date, quantity, and total amount.
4. Perform order insertion and modification operations.
5. Generate customer order history reports.

1. Database Design

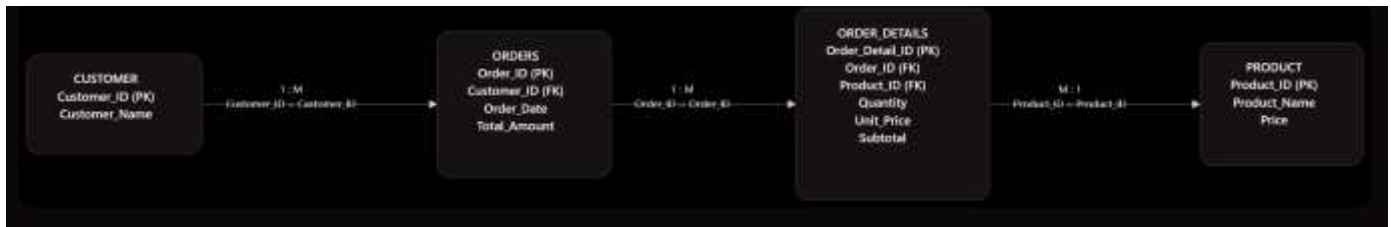
1.1 Orders Table

<i>Attribute</i>	<i>Data Type</i>	<i>Constraint</i>
<i>Order_ID</i>	INT	Primary Key
<i>Customer_ID</i>	INT	Foreign Key
<i>Order_Date</i>	DATE	NOT NULL
<i>Total_Amount</i>	NUMBER(10,2)	NOT NULL

1.2 Order_Details Table

<i>Attribute</i>	<i>Data Type</i>	<i>Constraint</i>
<i>Order_Detail_ID</i>	INT	Primary Key
<i>Order_ID</i>	INT	Foreign Key
<i>Product_ID</i>	INT	Foreign Key
<i>Quantity</i>	INT	NOT NULL
<i>Unit_Price</i>	NUMBER(10,2)	NOT NULL
<i>Subtotal</i>	NUMBER(10,2)	NOT NULL

2. Table Relationships



3. Table Creation

3.1 Create Orders Table

```
SQL> CREATE TABLE Orders (  
2     Order_ID      NUMBER(5) PRIMARY KEY,  
3     Customer_ID   NUMBER(5),  
4     Order_Date    DATE NOT NULL,  
5     Total_Amount  NUMBER(10,2) NOT NULL,  
6     CONSTRAINT fk_orders_customer  
7     FOREIGN KEY (Customer_ID)  
8     REFERENCES Customer(Customer_ID)  
9 );
```

Table created.

3.2 Create Order_Details

```
SQL> CREATE TABLE Order_Details (  
2     Order_Detail_ID  NUMBER(5) PRIMARY KEY,  
3     Order_ID         NUMBER(5),  
4     Product_ID       NUMBER(5),  
5     Quantity         NUMBER(5) NOT NULL,  
6     Unit_Price       NUMBER(10,2) NOT NULL,  
7     Subtotal         NUMBER(10,2) NOT NULL,  
8     CONSTRAINT fk_order_details_order  
9         FOREIGN KEY (Order_ID)  
10        REFERENCES Orders(Order_ID),  
11    CONSTRAINT fk_order_details_product  
12        FOREIGN KEY (Product_ID)  
13        REFERENCES Product(Product_ID)  
14 );
```

Table created.

4. Order Data Insertion

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (201, 101, SYSDATE, 2500.00);

1 row created.

SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (202, 102, TO_DATE('11-SEP-2026', 'DD-MON-YYYY'), 1800.00);

1 row created.

SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (203, 103, TO_DATE('12-SEP-2026', 'DD-MON-YYYY'), 3200.00);

1 row created.

SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (204, 104, TO_DATE('13-SEP-2026', 'DD-MON-YYYY'), 1500.00);

1 row created.
```

5. Order_Details Data Insertion

```
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (3, 202, 103, 2, 900.00, 1800.00);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (4, 203, 104, 2, 1600.00, 3200.00);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (5, 204, 101, 1, 1500.00, 1500.00);

1 row created.
```

6. Order Modification Operations

6.1 Modify Order Amount

```
SQL> UPDATE Orders
  2  SET Total_Amount = 2700.00
  3  WHERE Order_ID = 201;

1 row updated.
```

```
SQL> SELECT * FROM Orders
  2  WHERE Order_ID = 201;

ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
-----
      201          101 16-SEP-26          2700
```

6.2 Modify Order Date

```
SQL> UPDATE Orders
  2  SET Order_Date = TO_DATE('15-SEP-2026', 'DD-MON-YYYY')
  3  WHERE Order_ID = 202;

1 row updated.
```

7. Order_Details Modification

```
SQL> UPDATE Order_Details
  2  SET Quantity = 3,
  3      Subtotal = 3000.00
  4  WHERE Order_Detail_ID = 1;

1 row updated.
```

8. Customer Order History Report

```
SQL> SELECT
  2     c.Customer_ID,
  3     c.Customer_Name,
  4     o.Order_ID,
  5     o.Order_Date,
  6     od.Product_ID,
  7     od.Quantity,
  8     od.Unit_Price,
  9     od.Subtotal
 10 FROM Customer c
 11 JOIN Orders o
 12     ON c.Customer_ID = o.Customer_ID
 13 JOIN Order_Details od
 14     ON o.Order_ID = od.Order_ID
 15 ORDER BY c.Customer_ID, o.Order_Date;
```

CUSTOMER_ID

CUSTOMER_NAME

	ORDER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
	101					
Anitha Kumar	201	16-SEP-26	101	3	1000	3000
	101					
Anitha Kumar	201	16-SEP-26	102	1	500	500

9. Customer-wise Order Summary

```
SQL> SELECT
2      c.Customer_ID,
3      c.Customer_Name,
4      COUNT(o.Order_ID) AS Total_Orders,
5      SUM(o.Total_Amount) AS Total_Amount
6  FROM Customer c
7  JOIN Orders o
8    ON c.Customer_ID = o.Customer_ID
9  GROUP BY c.Customer_ID, c.Customer_Name
10 ORDER BY c.Customer_ID;
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_AMOUNT

101		
Anitha Kumar	1	2700
102		
Rahul	1	1800

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_AMOUNT

101		
Anitha Kumar	1	2700
102		
Rahul	1	1800

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_AMOUNT

103		
Priya	1	3200

104

Karthik S

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_AMOUNT

1	1500
---	------